**`total_steel_cost` = 2000 * int(state.quantity_of_steel)**

Yes! You've put the pieces together correctly. 🎉

Your expression uses:

- `int(state.quantity_of_steel)` to convert the player's input from a string to an integer.
- `2000 *` to multiply the quantity by the price per ton.
- `total_steel_cost` as the variable that stores the result.

One small syntax detail: in your actual Python code, don't put backticks around the variable name.

This follows the textbook's explanation that an assignment statement evaluates the expression on the right and stores its result in the variable on the left.

**Next question:** Once you've calculated `total_steel_cost`, what do you think you'll need to calculate for concrete?

the same things but question again: where do i put that line?

Put the line **inside your** `test_bridge` **function**, after the lines that save the TextBox values into `state`, and before the `return Page(...)` statement.

Think of the order:

1. Save the quantities from the TextBoxes to `state`.
2. Calculate the steel cost.
3. Calculate the concrete cost.
4. Display the next page with `return Page(...)`.

For example, the calculation belongs in the space just before `return Page(...)`—not inside `choose_materials`.

Does that help you see where it goes?

@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str, quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    return Page(state, ["Bridge Testing"
    ]) hows this?

You're making good progress! The steel calculation is in the right place: after saving the TextBox values and before `return Page(...)`.

Let's trace it with an example. If the user enters `"3"` for steel:

- `state.quantity_of_steel` contains the string `"3"`.
- `int(state.quantity_of_steel)` converts it to the integer `3`.
- Multiplying by `2000` produces the steel cost.

One important question: **Where is `total_steel_cost` stored right now?** Is it a local variable inside this function, or a field in `state`? That distinction will matter when you calculate the other material costs.

it is gonna be bad if I said it's a local variable?

Not bad at all! In fact, **a local variable is perfectly reasonable here.**

A local variable exists only while `test_bridge` is running. Since you're calculating the material costs inside that function, `total_steel_cost` can be local.

Here's something to think about as you continue:

If you calculate `total_concrete_cost` next, do you need to store it in `state` just to add it to `total_steel_cost` later in the same function? Why or why not?

need to store it in state

You could store it in `state`, but you don't necessarily need to!

Remember, both `total_steel_cost` and `total_concrete_cost` are calculated inside the same `test_bridge` function. A local variable can be accessed by other statements within that function.

Think about this: **If both costs are local variables, can you use the `+` operator to add them together before the function returns?**

Yes?

Yes! Exactly. You can use `+` to add two local variables together inside the same function.

Let's think one step ahead: once you've added all five material costs together, what should you call the variable that holds the combined cost?

ohhh ok so you're saying put all of them in the function before the return statement

Exactly! All five material cost calculations can go inside `test_bridge`, **after the TextBox values are saved and before the `return Page(...)` statement.**

Then, after calculating each individual cost, you'll add them together to get the total material cost.

You're understanding the structure now!

**One question:** Do you want to calculate the concrete cost next, using the same pattern you used for steel?

same pattern because didn't we figure out the exact numbers for all 5 materials?

Yes! We already established the prices for all five materials:

| Material | Cost per ton |
| --- | --- |
| Steel | $2,000 |
| Concrete | $100,000 |
| Stone | $45,000 |
| Asphalt | $4 |
| Aluminum | $8,500 |

You'll use the same pattern for each one:

- Multiply the material's cost per ton by its quantity.
- Convert the quantity from a string to an integer using `int()`.
- Store each result in its own local variable.

**Let's do concrete next:** What is the State attribute that holds the concrete quantity?

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    return Page(state, ["Bridge Testing"
    ]) thoughts
```

This is looking good! You've applied the same pattern consistently to all five materials.

Let's check your reasoning:

- Each quantity is converted from a `str` to an `int`.
- Each quantity is multiplied by its material's cost per ton.
- Each result is stored in a separate local variable.
- All five calculations are inside `test_bridge` and before `return Page(...)`.

Your next step is to combine those five costs into one total.

**Question:** What variable name would make sense for the sum of all five material costs? And which arithmetic operator will you use?

total_material_cost and id use +

Exactly! `total_material_cost` is a clear variable name, and `+` is the correct operator.

Now think about the expression you're building:

- Start with `total_steel_cost`.
- Add `total_concrete_cost`.
- Add the remaining three material costs.

**Where should you place the statement that calculates `total_material_cost`?** Remember the order of statements in your function.

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost +
total_aluminum_cost
    return Page(state, ["Bridge Testing"
    ]) like dis
```

Yes! That's the structure you were aiming for. Your `total_material_cost` calculation is in the right place: after all five individual costs and before `return Page(...)`.

Let's connect this to the textbook: Python evaluates the expression on the right side of `=` and stores the result in the variable on the left. Your addition expression follows that pattern.

**One thing to think about next:** Your starting budget is $600,000. Which arithmetic operator would you use to subtract `total_material_cost` from that budget?

-

Correct! `-` is the subtraction operator.

Now think about the two values involved:

- The starting budget: `600000`
- The total material cost: `total_material_cost`

What variable name would you choose to store the budget **after** subtracting the material cost?

leftover_budget

`leftover_budget` is a clear variable name! It describes what the value represents.

Now, before you write the assignment statement, think about this: should the calculation use the original starting budget (`600000`) or the current value of `state.remaining_budget`? Why?

The 2nd one?

Yes, that's a reasonable choice! `state.remaining_budget` holds the current budget in your program's state.

One thing to consider: is `state.remaining_budget` guaranteed to still be `$600,000` when this route runs, or could it already have been changed? That determines whether using it is appropriate.

wait so then it'd be starting budget

Exactly! If this is the first time the bridge is being tested, the starting budget is `$600,000`.

The distinction is:

- `starting_budget` represents the original amount.
- `state.remaining_budget` represents the amount currently available, which could change later.

Since you want to subtract the material cost from the original budget, which of those two names would make your calculation easiest to understand?

wait so it'd be leftover_budget - starting_budget

Almost! You have the order reversed.

Think about what you want mathematically:

**Starting budget − Total material cost = Leftover budget**

For example, if you start with $600,000 and spend $100,000, you should have $500,000 left.

So, which variable should be on the **left side** of the subtraction: `starting_budget` or `total_material_cost`?

oh sorry you know what i meant

Haha, yes, I got you! 😄 You want to subtract `total_material_cost` from `starting_budget` and store the result in `leftover_budget`.

Now, how would you express that as a Python assignment statement? Try writing it out!

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
```

```
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost +
total_aluminum_cost
    starting_budget - leftover_budget
    return Page(state, ["Bridge Testing"
    ])
```

You're very close! You've identified the two variables, but this line:
`starting_budget - leftover_budget`

is an expression by itself. Python calculates it, but doesn't store the result anywhere.

Remember the assignment pattern from your textbook: Python evaluates the right-hand expression and stores the result in the variable on the left.

Let's reason it out:

1. Which variable should receive the calculated result?
2. Which two values should be on the right side, and in what order?

Try adjusting that one line!

OHHH WAIT SORRY it's: leftover_budget = starting_budget - total_material_cost ?
YES!! 🎉 That's the assignment structure we were working toward!
`leftover_budget = starting_budget - total_material_cost`

Notice how:

- `leftover_budget` receives the result.
- `starting_budget` is the original amount.
- `total_material_cost` is what gets subtracted.

One small thing to check: have you already defined `starting_budget` somewhere in your program, or will you need to define it?

oh see I thought `starting_budget` was something already in there-

Ahh, I see where the confusion came from! No worries. 😊

You already have `state.remaining_budget` initialized to `600000` in your `State` setup, but `starting_budget` is not currently a variable you've shown me.

You have two concepts:

- `state.remaining_budget`: a value stored in your program's state.
- `starting_budget`: a name you'd need to define if you want to use it separately.

**One question:** Would you rather use your existing `state.remaining_budget` or create a separate `starting_budget` variable? Both approaches are possible, but let's choose one and keep your code consistent.

well which one makes more sense

I'd recommend using your existing `state.remaining_budget`. It already starts at `$600,000`, so you don't need to introduce another variable just to represent the starting amount.

That also keeps your code consistent with the `State` structure you've already built.

One important distinction: since you want to preserve the original budget for later comparisons, think about whether you should **update** `state.remaining_budget` directly or keep the original amount separate.

Which behavior do you want: should the budget permanently decrease after testing, or should it always be calculated fresh from $600,000?

What do you mean

I mean: **what should happen if the user tests the bridge more than once?**

There are two possibilities:

1. **Budget decreases permanently:** After the first test, the remaining budget becomes lower. If they test again, the next cost is subtracted from that smaller amount.
2. **Budget always starts at $600,000:** Every test calculates the leftover budget using the original $600,000, regardless of previous tests.

Since your `state.remaining_budget` already stores the budget, I'm asking whether it should represent the **current amount left** or the **original starting amount**.

Which behavior do you want?

the 2nd one makes more sense

That makes sense! Then you want every test to calculate the leftover budget from the **original $600,000**, rather than from a previously reduced amount.

Since `state.remaining_budget` is currently initialized to `600000`, think about this distinction:

- `state.remaining_budget` — the amount you want to display as left over after testing.
- Original starting budget — the fixed amount you subtract the material cost from each time.

**One question:** Would you be comfortable adding a separate variable to represent the original starting budget, so the value never changes when you update the remaining budget?

wait hold on- which one do u think is mentally easier?

Honestly, I think **the second approach is mentally easier for your project**: always calculate from the original $600,000.

Why?

- You have one fixed starting budget.

- You calculate the total material cost.
- You subtract that cost from $600,000.
- You store the result as the remaining budget.

You don't have to worry about accidentally subtracting the cost twice or carrying over an old budget value.

I'd keep the original budget separate from `state.remaining_budget`. That way, each variable has one clear purpose.

**Does that distinction make sense?** One variable remembers where you started; the other stores what you have left.

right yeah no it does so what do I need to do

You'll need to make **two small changes** to how you represent the budget.

**1. Add a separate starting budget variable**

You already have `state.remaining_budget = 600000`.

You'll want another variable that holds the original `$600,000` and never changes.

Think of it this way:

- `starting_budget` = always $600,000
- `state.remaining_budget` = calculated leftover money

**2. Calculate the leftover budget in `test_bridge`**

You already have this expression:

```
total_material_cost
```

Now think about the subtraction:

- What value should you subtract from?
- What value should you subtract?

You previously came up with:

```
leftover_budget = starting_budget - total_material_cost
```

That same reasoning applies here!

**Your turn:** Where do you think the `starting_budget` variable should be initialized: alongside your other initial state values, or inside the `test_bridge` function? Why?

im confused on where I even need to *put it*

Totally fair! Let's locate it in your existing code.

You have two places to think about:

**1. Where your starting budget belongs**

Put it with your other initial values, where you currently initialize `bridging_off` and set `remaining_budget=600000`.

That way, the original budget is established once when the program starts—not every time someone tests the bridge.

**2. Where your leftover budget calculation belongs**

Put that inside `test_bridge`, after you calculate `total_material_cost` and before the `return Page(...)` statement.

So the order inside `test_bridge` is:

1. Save the quantities entered by the user.
2. Calculate each material's cost.
3. Calculate `total_material_cost`.
4. Calculate the leftover budget.
5. Return the results page.

**Quick question:** Do you see where `bridging_off = State(...)` is created in your code? That's the first place I want you to look.

ohhh ok so you want me to say starting_bridge = 600000 and put that in the bridging_off variable

Almost! You're thinking in the right direction. 😊

One small distinction:

- `starting_budget` should hold the original value `600000`.
- `bridging_off = State(...)` is where you initialize the attributes belonging to your program's state.

You don't want to replace `bridging_off` with `600000`, because `bridging_off` represents your entire State object.

**Think about this:** If you wanted `starting_budget` to be accessible throughout your program, would it make sense to add it as another attribute inside `State(...)`, just like `remaining_budget`?

That would keep your budget information organized in one place.

Yes?

Yes! Exactly. Add `starting_budget` as another attribute in your `State` structure, right alongside `remaining_budget`.

That way:

- `starting_budget` stays at `600000`.
- `remaining_budget` gets updated with the calculated leftover amount.

**One thing to think about:** If you add a new attribute to your `State` dataclass, what other place in your code will need to include that attribute when creating the `State` object?

```
@dataclass
class State:
    selected_material: list[str]
    remaining_budget: int
    starting_budget = int
    optional_features: list[str]
    current_bridge: Bridge
    name_of_bridge: str
    list_of_criteria: list[Criteria]
    quantity_of_steel: str
    quantity_of_concrete: str
    quantity_of_stone: str
    quantity_of_asphalt: str
    quantity_of_aluminum: str
    quantity_of_timber: str
    quantity_of_rubber: str
    quantity_of_fiber_reinforced_polymers: str
```

so that's how that'd look?

You're very close! The placement is exactly right. There's just one small syntax detail to fix.

Look at these two lines:

remaining_budget: int
starting_budget = int

Notice how your other State attributes use a **colon (`:`)** between the variable name and its type.

In a dataclass, that colon tells Python the attribute's type.

So, `starting_budget` should follow the same pattern as `remaining_budget`.

**One more step:** After you add that attribute to `State`, remember to provide its initial value when you create `bridging_off = State(...)`.

Can you spot which punctuation is different between those two lines?

fixed it now what
Great! Now let's go to where you create your State object:
bridging_off = State(...)

Remember, this is where you give each State attribute its starting value.

You already have:

- `remaining_budget` initialized to `600000`.
- Now you need to initialize `starting_budget` to `600000` as well.

**Your turn:** Add that new attribute to your `bridging_off = State(...)` call.

Once you've done that, we'll move back to `test_bridge` and calculate the leftover budget.

quantity = Criteria(budget = 600000, time_duration = 365.0, construction_workers = 100)
starting_bridge = Bridge(length = 1000, width = 110 , qualities = [quantity] , stability = True)
bridging_off = State(selected_material = None, remaining_budget = 600000, starting_budget = 600000, optional_features = [], current_bridge = starting_bridge , name_of_bridge = "", list_of_criteria = [quantity]) better?\

Yes! This is much better. You're keeping both budget values in your `State` object, and you've initialized them to the same original amount.

Let's trace what you've set up:

- `remaining_budget = 600000` — initial amount available.
- `starting_budget = 600000` — original amount that stays fixed.

One thing to check: did you also add `starting_budget: int` inside your `State` dataclass? Python needs that attribute defined there before you can pass it into `State(...)`.

The textbook's **1B1 Variables Reading** explains that clear variable names help communicate what each value represents. You're applying that idea here!

Yes I did

Perfect! Then your budget setup is ready.

Now let's return to your `test_bridge` function.

You already calculate `total_material_cost`. The next step is to calculate the leftover budget using the two variables we just established.

Remember the expression you came up with earlier?

```
leftover_budget = starting_budget - total_material_cost
```

**Your turn:** Where should the result of that calculation be stored: in a temporary local variable, or in `state.remaining_budget`? Think about which one the Results page will need to access.

The 2nd one?

Yes! The second option: `state.remaining_budget`.

That way, the updated budget becomes part of your `State` object and can be accessed by your Results page.

One thing to think about: **Should `state.starting_budget` change when you update `state.remaining_budget`?** Remember, you wanted the calculation to always start from the original $600,000.

The textbook's **1B2 Tracing Reading** explains how assignment statements update a variable's value.

Yes it should

Let's think through that carefully. You originally wanted `starting_budget` to stay at $600,000 so every bridge test calculates from the original amount.

If you change `starting_budget` too, what happens when you test the bridge a second time?

**Which variable should stay fixed, and which variable should change?**

ok nevermind it shouldn't

Exactly! Keep `state.starting_budget` fixed at **$600,000** and update only `state.remaining_budget`.

That way, every test calculates the leftover amount from the original budget rather than subtracting from an already reduced amount.

**Next question:** In your `test_bridge` function, where would you place the budget calculation: before or after `total_material_cost` is calculated?

The textbook's **1B2 Tracing Reading** explains why statement order matters: Python evaluates statements in sequence, so a variable must have a value before you use it.

after `total_material_cost` is calculated

Exactly! Python needs `total_material_cost` to have a value before you can use it in the budget calculation.

Now, think about the assignment statement:

- Which object contains the budget values?
- Which attribute holds the original budget?
- Which attribute should receive the newly calculated leftover budget?

Try writing that one assignment statement yourself, and I'll help you check your reasoning.

Remember **1B2: Tracing Reading**—the right-hand expression is evaluated before its result is assigned to the left-hand variable.

wait hold on- doesn't total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost + total_aluminum_cost
?

Yes! That's exactly what your `total_material_cost` statement does.

Let's break it down:

- `total_steel_cost`, `total_concrete_cost`, etc. are the individual material costs.
- The `+` operators add those five costs together.
- The result is assigned to `total_material_cost`.

So after that statement runs, `total_material_cost` contains the combined cost of all five materials.

**Here's my question:** Does that statement calculate the leftover budget, or does it calculate how much you spent?

Think about the difference between *total cost* and *remaining budget*.

how much was spent

Exactly! `total_material_cost` represents how much money was **spent**.

To find how much money is left, you need to subtract that amount from the original budget.

Think about this distinction:

- `starting_budget` = money available at the beginning.
- `total_material_cost` = money spent.

- `remaining_budget` = money left.

**Your turn:** Which arithmetic operator would you use to calculate the remaining budget?

As described in the textbook's **1B2 Tracing Reading**, Python evaluates the expression before assigning its result to a variable.

isnt it this?  leftover_budget = starting_budget - total_material_cost

Yes! That's the subtraction expression you need. You're subtracting the total amount spent from the original budget.

There's just one detail to consider: `leftover_budget` is a separate local variable, while your Results page needs to access the value through `state`.

How could you make sure the calculated value is stored in `state.remaining_budget` instead?

Think back to the textbook's **1B2 Variables Reading**: the variable on the left side of `=` is the one being updated.

so `state.remaining_budget` is one on the left

Yes! `state.remaining_budget` should be on the **left side** because that's the attribute you want to update.

Now think about the right side: which two values do you need to subtract to calculate the remaining budget?

You're very close! Remember, `state.starting_budget` holds the original amount.

ok so on the right it'd be state.starting_budget - yeah idk what goes here

You're missing just one piece: the variable that holds **how much was spent**.

Look at the statement you already wrote:

```
total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost + total_aluminum_cost
```

What variable name appears on the left side of that statement? That's the value you need on the right side of your budget subtraction.

So, what do you think goes after the minus sign?

(See **1B2: Tracing Reading**—you can read a variable by using its name in an expression.)

oh so total_material_cost

Yes! Exactly. `total_material_cost` is the amount spent on materials.

So your subtraction uses:

- `state.starting_budget` — original budget
- `total_material_cost` — amount spent

Now put those together on the right side of your assignment statement. What do you get?

You're applying the **1B2 Tracing Reading** idea: read existing variable values to calculate a new value, then assign it to the variable on the left.

`state.starting_budget - total_material_cost`

Yes! Now put that subtraction expression on the **right side** of your assignment, with `state.remaining_budget` on the left.

One quick tracing question: if the starting budget is `600000` and materials cost `250000`, what value should `state.remaining_budget` hold after the assignment runs?

350000

Exactly! $600,000 - 250,000 = 350,000$.

That means `state.remaining_budget` will hold `350000` after the assignment.

**Next question:** Where in your `test_bridge` function should this budget calculation go—before or after the `total_material_cost` calculation? Remember, Python executes statements from top to bottom (**1B3: Modules Reading**).

Before?

Almost! Think about what Python needs before it can subtract.

If the budget calculation runs **before** `total_material_cost` is calculated, does Python already know the value of `total_material_cost`?

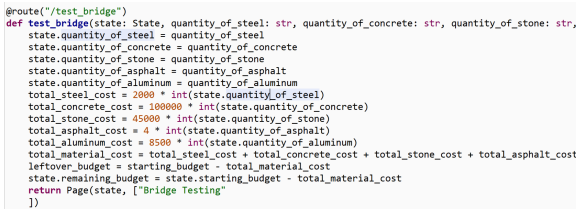
So should the budget calculation go before or after it?

After

Exactly! The `total_material_cost` must be calculated first so Python has a value to subtract.

Place your budget assignment **immediately after** the `total_material_cost` calculation and before `return Page(...)`.

What variable will hold the new budget amount?



```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost
    leftover_budget = starting_budget - total_material_cost
    state.remaining_budget = state.starting_budget - total_material_cost
    return Page(state, ["Bridge Testing"
    ])
```

hold up this is what i have now where do you want me to put it?

You already put it in the right place! Look at these two lines in your screenshot:

- `leftover_budget = ...`
- `state.remaining_budget = ...`

The **second line** is the one that stores the result in your program's shared `State`, so your Results page can access it later.

The first line is redundant, and `starting_budget` there isn't referenced through `state`.

**Your next step:** Remove the first line, then keep the `state.remaining_budget` line right where it is—directly after `total_material_cost` and before `return Page(...)`.

That follows **1B2: Tracing Reading**: the assignment updates the variable on the left.

Remove what first line?
I mean this line in your screenshot:
leftover_budget = starting_budget - total_material_cost

It's the line **directly above** `state.remaining_budget = ...`.

You don't need both lines because you're storing the calculated budget in `state.remaining_budget` already.

Can you see that first line now?

Did that now what?

Great! Now let's check that the calculation works before moving on.

In your `test_bridge` function, temporarily trace these example values:

- `state.starting_budget` = `600000`
- `total_material_cost` = `200000`

**Question:** What value should `state.remaining_budget` contain after this line runs?

This helps us verify the subtraction before we use that value on your Results page.

400000

Exactly! `state.remaining_budget` should be `400000` in that example.

Your budget calculation is in place. Now let's move to the next part: displaying the remaining budget on your Results page.

**Have you started writing your `results` route yet?**

I have not but answer me this without  ANY.OTHER.SIDE QUESTIONS: is the results route gonna be easy, medium or hard to figure out? because I feel like now is a good time to bring up other parts that I need to implement in the code. but answer me that first

I'd say **easy**. The Results route should be one of the easier parts because its main job is to display information your program has already calculated or stored, such as `state.remaining_budget`.

You don't need to worry about it being a major coding challenge. **Now is a good time to bring up the other parts you need to implement** before we build it.

ok so 1: I have to implement 3 if statements: 1st if statement: if the total cost exceeds the $600,000 budget, the player must rethink about what materials they selected and possibly remove some that fits the budget while still keeping stability in check

2nd if statement: if the bridge fails being stable, the player must restart the bridge over entirely or improve their design choices that may've caused it to be unstable in the first place

3rd if statement: if a material is selected by the player, the game then determines its cost and adjusts the remaining budget ; next, I have to implement 1 for loop: Loop:

for criteria in bridge.qualities:

#loop iterates through list of Criteria instances stored in class Bridge qualities attribute; allows program to work on budget, time duration and number of construction workers

Got it. These requirements add **three conditional decisions (`if` statements)** and **one loop (`for` statement)** to your simulator. Let's connect each one to the code structure you already have.

**1. The three `if` statements**

| Condition | What the program needs to determine |
|---|---|
| **Budget exceeded** | Is the total material cost greater than the starting budget? If so, the player needs to reconsider material choices while keeping bridge stability in mind. |
| **Bridge unstable** | Is the bridge's stability value indicating failure? If so, the player needs an opportunity to restart or improve the design. |
| **Material selected** | Has the player selected a material? If so, the program needs to account for that material's cost and update the remaining budget. |

One important distinction: your existing `test_bridge` function already calculates the total material cost and remaining budget. The new conditions should determine **what happens based on those values**, rather than simply repeating the calculations.

**2. The `for` loop**

Your loop:

for criteria in bridge.qualities:

This visits each `Criteria` instance stored in the bridge's `qualities` list, one at a time.

According to your `Criteria` class, each instance contains three attributes:

- `budget`
- `time_duration`

- `construction_workers`

That means the loop gives your program a way to examine each set of criteria associated with the bridge.

**3. How the loop relates to the conditions**

The loop and the `if` statements have different jobs:

- The **loop** visits the bridge's criteria.
- The **if statements** make decisions when particular conditions are met.

For example, your loop variable `criteria` represents one `Criteria` object during each iteration. You can then examine its attributes individually.

One thing to keep in mind: your loop only iterates through the criteria you have stored. It does not automatically check whether the bridge meets them; you'll need to think about what comparisons your program should make.

**4. A useful order to think about**

The textbook's **1B3: Modules Reading** explains that Python executes statements sequentially, from top to bottom. That matters here because calculations and decisions depend on values being available first.

A sensible conceptual order is:

1. Determine the selected materials and their costs.
2. Calculate the total material cost and remaining budget.
3. Check whether the budget has been exceeded.
4. Examine the bridge's stability.
5. Evaluate the bridge criteria using the loop.

You may adjust the order as your design develops, but remember the principle from **1B2: Tracing Reading**: a variable must have a value assigned before your program reads it.

You now have a clear distinction between the calculations, conditional decisions, and criteria loop.

Ok so you just threw a lot of things at me just now. Break it down, what do I need to do first

Let's do **just one thing first: the budget `if` statement.**

You already calculate:

- `total_material_cost`
- `state.remaining_budget`

So you already have the values needed for this condition.

**Your first task:** Think about what comparison Python needs to make to determine whether the player has gone over budget.

For example, should it check whether `total_material_cost` is **greater than**, **less than**, or **equal to** `$600,000`?

The textbook's *1A6: Math Reading* covers comparison-related math concepts, and *1B2: Tracing Reading* explains how Python evaluates expressions. Let's focus on understanding that comparison before moving on.

it should check ONLY IF the total_material_cost is greater than 600,000

Exactly. The condition should use the **greater-than operator (`>`)**, because you only want the budget warning to happen when the total cost exceeds $600,000.

Now think about what should happen *inside* that `if` statement: what message or action should the player see when their material costs exceed the budget?

This connects to the textbook's **1A6: Math Reading**, which covers arithmetic operators, and **1B3: Modules Reading**, which explains how Python executes statements in order.

if they go over budget, that means they must reduce their selections on materials

Right. So the purpose of your first `if` statement is to **detect when the player exceeds the budget and prompt them to reduce their material quantities.**

There's one important distinction:

- The condition detects whether the budget is exceeded.
- The response gives the player a chance to reduce material selections.

You don't need to implement the material-reduction process yet.

This follows the *1B3: Modules Reading* concept: Python executes the statements inside an `if` block only when its condition evaluates to `True`.

**For now, focus on what the player should see when they go over budget.**

well, I didn't think that far into it, what do *you* think would be easy enough for when the player sees that?

I'd keep it simple: display a message telling the player they're over budget and need to go back to the materials page to adjust their quantities.

For example:

> **Over Budget!**
>
> Your selected materials exceed the $600,000 budget. Please return to the materials page and reduce the quantities of one or more materials before continuing.

Then give them a **"Return to Materials"** button.

That way, you don't have to build a complicated automatic material-removal system. The player makes the adjustments themselves.

This is a manageable first version for a CS1 project, and it keeps the player in control of their design.

yes i agree- is that going to be a hassle to make?

No, not at all! I'd consider this **easy to moderate**, especially since you already have your materials page and budget calculations working.

You'll mainly need to:

1. Detect when the total cost exceeds $600,000.
2. Display the over-budget message.
3. Add a button that takes the player back to the materials page.

The only slightly tricky part is making sure the updated material quantities are saved when the player returns.

But we can work through that one piece at a time. No need to build everything at once!

Ok so lets get started

Great! Let's build this one piece at a time.

**Step 1: Find where the budget check belongs.**

In your `test_bridge` function, you already calculate:

- `total_material_cost`
- `state.remaining_budget`

The budget check needs to happen **after those calculations and before the `return Page(...)` statement**.

We aren't writing the whole condition yet.

First, let's establish what the `if` statement will evaluate:

- `total_material_cost` is the actual cost of the materials.
- `600000` is the maximum allowed budget.
- `>` checks whether the actual cost exceeds that limit.

This uses the comparison concepts from your textbook's *1A6: Math Reading*.

**Your turn:** What Python comparison operator would you use to check whether `total_material_cost` exceeds `600000`?

>

Exactly! `>` means **greater than**.

Now let's think about the next part.

If that condition evaluates to `True`, Python will execute the indented statements inside the `if` block.

What message would you like the player to see when they're over budget? You can use the wording we came up with earlier.

I want it to say: You have went over budget. Please go back and find a better solution - there short and simple

That works! It's short, clear, and tells the player what to do.

One small grammar improvement: **"You have gone over budget. Please go back and find a better solution."**

For now, that's the message you want displayed inside your budget `if` block.

In Python, remember that the `print()` function displays a message in the console, but your Drafter app uses page elements to display messages to players.

**Next, we'll figure out how to display that message on your page.**

Ok how do I

Since you're using **Drafter**, you'll want to display the message as a page element, rather than using `print()`.

Let's look at a small example unrelated to your assignment:

```
return Page(state, [
    "Your order is over budget!"
])
```

The string inside `Page` becomes visible on the player's page.

For your simulator, the message belongs in the page returned when the budget condition is `True`.

**One important concept:** Your normal bridge-testing page and your over-budget message are two different possible outputs.

Your textbook's *1A3: Console I/O Reading* explains that output is how a program presents processed information to the user.

ok so do you or do you not want me to put an example like that in my bridge_testing part of the code?

Yes! Put the **over-budget message and the "Return to Materials" button** in the `Page` returned when your budget condition is `True`.

Don't put that example in as a separate, unconditional page—it was only showing you how a page displays text.

Your existing `"Bridge Testing"` page should be the output for the other situation, when the player is not over budget.

In *1A3: Console I/O Reading*, your textbook explains how a program presents output to the user. Here, Drafter's `Page` is how you display that output.

ok so wait wait wait- here's how I'm thinking of the if statement: if state.remaining_budget > 600000: what now?

You're thinking in the right direction by using an `if` statement, but let's check the variable first.

You previously calculated:

```
state.remaining_budget = state.starting_budget - total_material_cost
```

So `state.remaining_budget` represents the **money left**, not the amount spent.

If the player has gone over budget, would the remaining budget be **greater than** or **less than zero**?

Think about that comparison before writing the condition.

Greater than

Almost! Remember, `state.remaining_budget` is the money **left over**.

If the player spends more than $600,000, the remaining budget becomes negative.

For example, if they have $50,000 left, that's positive. If they're $50,000 over budget, that's negative.

So, should your comparison be `< 0` or `> 0`?

Think about *1A7: Logic Reading* in your textbook: comparison operators evaluate to `True` or `False`.

Ok other one then

Yes! The comparison should be `< 0`.

That means the remaining budget is negative, so the player has spent more than the starting budget.

Now, think about what should happen inside that `if` block: which two things do you want the player to see or be able to click?

i only want them to see the message and the “Return to Materials” button

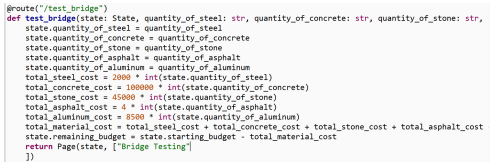
Perfect. Then your `if` block only needs to return a page containing those two elements:

1. The message: `"You have gone over budget. Please go back and find a better solution."`
2. The button: `"Return to Materials"`, which navigates to `"choose_materials"`.

Put that page inside the `if` block in your `test_bridge` function, after the budget calculation.

**One important detail:** Your normal `"Bridge Testing"` page should only be returned when the player is *not* over budget. Otherwise, the player could see the wrong page.

Your textbook's *1B3: Tracing Reading* explains that Python executes statements sequentially, so the placement of your `return` statements matters.



```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost
    state.remaining_budget = state.starting_budget - total_material_cost
    return Page(state, ["Bridge Testing"
    ])
```

ok this is not helping- how do I even format my if statement and where do I even put it in here

You're right—I was being too vague. Let's use the exact layout of your function and focus on **where the indentation goes**.

Look at the last three parts of your code:

1. **Budget calculation:** Keep `state.remaining_budget = ...` exactly where it is.

2. **Your new `if` statement:** Put it immediately below that budget calculation, at the same indentation level as your current `return Page(...)`.
3. **Inside the `if`:** Indent one level (4 spaces). This is where the page containing your over-budget message and `"Return to Materials"` button goes.
4. **Normal bridge-testing page:** Keep your existing `"Bridge Testing"` return below the `if`, aligned with the `if` itself. That way, it is reached when the condition is false.

The structure is important: Python uses indentation to determine which statements belong inside an `if` block. Your textbook explains this in **1B3: Tracing Reading**, where it discusses how Python executes statements in order.

**Your next step:** Try adding just the `if` line and its colon beneath the budget calculation. Don't worry about the message or button yet. Show me that part, and we'll check the indentation together.

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost
    state.remaining_budget = state.starting_budget - total_material_cost
    if:
        "Return to Materials"
    return Page(state, ["Bridge Testing"
    ])
```

ok so this

You're getting closer! There are **three things** to fix in this attempt:

1. **Your `if` line is incomplete.** Python needs a Boolean condition between `if` and the colon. You already identified that the condition should check whether the remaining budget is below zero.

2. **Your button text is currently just a string.** `"Return to Materials"` by itself won't create a clickable button. Drafter uses a `Button` component for that.

3. **Your message and button need to be inside a `Page` return.** They should be together in the page displayed when the condition is true.

Your textbook's *1A7: Logic Reading* explains that an `if` statement uses a Boolean expression to decide whether its indented statements execute.

**Let's fix one thing at a time:** Can you add the Boolean condition to your `if` line first? Leave the rest alone for now.

so again- what do you want me to put it in the if statement itself
Put this condition on the `if` line:
if state.remaining_budget < 0:

That's it for the condition!

- `state.remaining_budget` is the money left.
- `< 0` checks whether that amount is negative.
- `:` starts the indented block.

Your textbook's *1A7: Logic Reading* covers the `<` comparison operator.

if state.remaining_budget > 0:
    "Return to Materials"

You're very close, but there's one important logic issue.

`state.remaining_budget > 0` means the player **still has money left**. That means they're under budget.

For your over-budget condition, think about the opposite: the remaining budget is negative.

Also, `"Return to Materials"` by itself is only text, not a clickable button. Drafter uses `Button()` to create clickable buttons.

**First, change your comparison operator.** Which operator makes the remaining budget negative: `<` or `>`?

```
if state.remaining_budget < 0:
    Button("Return to Materials")
```

Yes! Your `if` condition and indentation are now right.

One thing is missing: Drafter's `Button` needs **two arguments**:

- The text the player sees: `"Return to Materials"`
- The route it should navigate to: `"choose_materials"`

Also, the button needs to be inside a `Page` returned by your function, not standing alone.

Try putting those pieces together. Remember, the over-budget message should appear on that same page.

oh yeah thats right I need to put that message in there, so where in my code do I put that message?

Put the message **inside the `if` branch**, right before the button. Both should be part of the list passed to `Page(...)`.

Think of the structure this way:

1. The `if` checks whether the remaining budget is negative.
2. If it is, the page displays your message.
3. The button appears underneath the message.

In Drafter, text inside a `Page` list is displayed to the player. (See the textbook's **1A3 Console I/O Reading** for the distinction between program output and the information a user sees.)

What do you think needs to wrap around both the message and the button so they appear on the same page?

Page?

Yes! `Page(...)` is the Drafter structure that displays content to the player.

Your `return Page(...)` currently displays `"Bridge Testing"`. For the over-budget case, you'll need a separate `return Page(...)` inside your `if` statement.

The message and button both go inside that page's list, like the other text and buttons you've already used in your routes.

```
if state.remaining_budget < 0:
    return (state, [("You have went over budget. Please go back and find a better solution"), Button(
return Page(state, ["Bridge Testing"
])
```

grey stuff again

I see what you mean! The grey background looks like an editor highlight or selection, not necessarily an error in your Python code.

But I notice one important difference in your code:

- Your normal page starts with `return Page(state, [...])`.
- Your over-budget page starts with `return (state, [...])`.

**What word is missing after `return` in your over-budget branch?**

That distinction matters because `Page` creates the Drafter page.

True?

Not quite! The missing word is `Page`, not `True`.

Remember:

- `if` checks a Boolean condition (`True` or `False`).
- `Page(...)` creates the page that Drafter displays.

So, what should come immediately after `return` in that branch?

Dunno
No worries! Look at your working line below:
return Page(state, ["Bridge Testing"])

The over-budget branch needs that same `Page` structure. You're returning a page to the player, not just a pair of values.

Try adding `Page` after `return` in your over-budget branch. Then you'll have the same structure in both branches.

(Your textbook's **1B3 Modules Reading** also reminds us that Python executes statements in order, so the first `return` handles the over-budget case before the normal page return.)

if state.remaining_budget < 0:
    return Page (state, [("You have went over budget. Please go back and find a better solution"), Button("Return to Materials", "/choose_materials"),
  return Page(state, ["Bridge Testing"
  ])
ok so i edited a little bit of it and now there's more grey stuff

You're getting closer! The grey highlighting itself still looks like an editor highlight, not necessarily an error.

I do see a Python syntax issue, though: **your first `return Page(...)` hasn't been closed yet.**

Look at this part:

Button("Return to Materials", "/choose_materials"),

After the button, Python still needs the closing symbols for:

- The list `[...]`
- The `Page(...)` call

Those need to be closed **before** the second `return Page(...)` begins.

Also, based on your earlier route, check whether the button should use `"choose_materials"` rather than `"/choose_materials"`.

Your textbook's **1B2 Tracing Reading** discusses how Python evaluates statements, but here we're checking the structure of the expression first.

Can you spot where those two closing symbols belong?

nope can you please just tell me

Absolutely! You need to add `] )` after your button (without the space between the closing symbols).

So the end of your first return should look like this:

Button("Return to Materials", "choose_materials")])

That closes:

- `]` — the list of page elements.
- `)` — the `Page(...)` call.

Then your second `return Page(...)` stays where it is, aligned with the first `return`.

The button route should be `"choose_materials"`, matching the route name you defined earlier.

ok so now time for my 2nd if statement?

Yes! Let's tackle the second `if` statement: **checking whether the bridge is stable.**

Before writing anything, let's think about the logic.

Your `Bridge` dataclass has an attribute called `stability`, which is a Boolean (`True` or `False`).

If the bridge is unstable, what value do you expect `stability` to have: `True` or `False`?

False

Exactly! If the bridge is unstable, `stability` is `False`.

Now think about where this condition belongs: should the stability check happen **after the budget check**, so the program only evaluates stability if the player is within budget?

Your textbook's **1A7 Logic Reading** explains how Boolean values (`True` and `False`) are used in conditional decisions.

yeah see i figured it should be checked at the results page

Yes! That makes sense. The **Results page** is where the player should see the final outcome of the bridge test.

One distinction to keep in mind:

- `test_bridge` can perform the stability check.
- `results` can display whether the bridge passed or failed, and explain what the player should do next.

That keeps the processing separate from the output, which matches the **Input → Processing → Output** model in your textbook's *1A3 Console I/O Reading*.

So let's keep the stability decision in mind as we build toward the Results page.

ok so where do you suggest it'd be put

I suggest putting the **stability check in** `test_bridge`, after the budget check.

That way, the program follows this order:

1. Calculate the material costs and remaining budget.
2. Check whether the player is over budget. If so, send them back to materials.
3. If they're within budget, check `state.current_bridge.stability`.
4. Send the player to the Results page, where they see whether the bridge passed or failed.

The important distinction is that `test_bridge` handles the **decision**, while `results` displays the **outcome**. This follows the Input → Processing → Output model in your textbook's *1A3 Console I/O Reading*.

Since you said an unstable bridge has `stability = False`, what Boolean condition would identify that case?

What do you mean

I mean: in your `test_bridge` function, **after the budget check**, you would check whether the bridge is stable.

You told me that `stability` is:

- `True` = bridge is stable
- `False` = bridge is unstable

So the condition needs to check whether `state.current_bridge.stability` is `False`.

In Python, what comparison operator would you use to check whether a Boolean value is `False`?

no no no- i want it to be stable- if it's unstable, they gotta start over

Oh, got it! You want the bridge to **pass the stability check** to continue. If it's unstable, the player must start over.

So the logic should be:

- If `state.current_bridge.stability` is `False`, send the player to restart the bridge.
- Otherwise, let the player continue to the Results page.

That means your `if` condition checks for the unstable case. The textbook's *1A7 Logic Reading* explains that Boolean conditions evaluate to either `True` or `False`.

**Where should the restart send the player: back to `bridge_building` or all the way back to the beginning?**

no bring em back to bridge_building

Perfect! Then your stability check belongs in `test_bridge`, after the budget check.

If the bridge is unstable (`stability` is `False`), the player should be sent back to the `bridge_building` route. If it is stable (`True`), they can continue to Results.

For the restart button, the route name is `"bridge_building"`—the same route name you used when defining that page.

ok so how do i put that together code-wise?

You'll add a **second `if` statement** in your `test_bridge` function, immediately after your budget check.

I won't write the whole assignment code for you, but here's the structure to build:

```
if state.current_bridge.stability == False:
    return Page(state, [
        "Your bridge is unstable! You must redesign it.",
        Button("Restart Bridge", "bridge_building")
    ])
```

Put this before your normal Results page return. The normal return should stay at the same indentation level as the `if`.

One concept from your textbook's *1A7 Logic Reading*: the comparison checks a Boolean condition, which evaluates to `True` or `False`.

**Your turn:** Why do you think this condition checks for `False` rather than `True`?

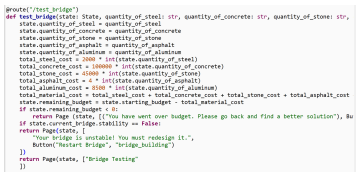
no hold on here's this: where do i put that in test_bridge? after my first if statement? Where??

Yes! **Put the second `if` immediately after the first budget `if` block**, but before your normal `return Page(state, ["Bridge Testing"])`.

Your order inside `test_bridge` should be:

1. Calculate `total_material_cost`.
2. Calculate `state.remaining_budget`.
3. First `if`: check whether the player is over budget and return them to materials.
4. Second `if`: check whether the bridge is unstable and return them to `bridge_building`.
5. Normal `return Page(...)`: send the player to the Results page if neither condition applies.

**Important:** The second `if` must be outside the first `if` block, at the same indentation level.



how's this looking

You're very close! The overall order is what we want:

1. Calculate material costs.
2. Calculate remaining budget.
3. Check whether the player is over budget.
4. Check whether the bridge is unstable.
5. Continue if both checks pass.

**I see one indentation issue to fix.**

Your second `return Page(...)` needs to be indented **one level (4 spaces) inside** this line:

if state.current_bridge.stability == False:

Your final `return Page(state, ["Bridge Testing"])` should stay aligned with the two `if` statements, outside both blocks.

That way, Python only displays the restart message when the bridge is unstable.

This connects to your textbook's *1B3 Modules Reading*: Python executes statements sequentially, and indentation determines which statements belong to a conditional block.

Fix that indentation and show me the updated version!

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost +
total_aluminum_cost
    state.remaining_budget = state.starting_budget - total_material_cost
    if state.remaining_budget < 0:
        return Page (state, [("You have went over budget. Please go back and find a better solution"), Button("Return to
Materials", "choose_materials")]),
    if state.current_bridge.stability == False:
        return Page(state, [
```

```
        "Your bridge is unstable! You must redesign it.", Button("Restart Bridge", "bridge_building")
        ])
    return Page(state, ["Bridge Testing"
    ])
```

Your indentation is looking good now! Both `if` statements are at the correct level, and the stability check is in the right place.

I spotted **one small syntax issue** in your first `return Page` statement.

Look at the very end of this line:

return Page (state, [("You have went over budget. Please go back and find a better solution"), Button("Return to Materials", "choose_materials")]),

Notice the comma after the final `)`?

**Remove that comma.** In Python, a trailing comma there makes the return value a one-element tuple instead of a `Page` object.

Your second `return Page` and final `return Page` are positioned correctly.

This is a useful connection to the textbook's *1A8 Interactive Evaluation Reading*: Python expressions evaluate to values, and the type of the returned value matters.

Make that one change. Then we can look at what happens when the bridge is stable!

ok done now time for the if it *is* stable part

Good! Here's an important Python concept: **you may not need another `if` statement at all.**

Look at your existing logic:

- If stability is `False`, Python enters the restart branch and returns to `bridge_building`.
- If stability is `True`, Python skips that branch and continues to the next statement.

That means your final `return Page(...)` can handle the stable bridge case.

Right now, it displays `"Bridge Testing"`, but you said you want stable bridges to go to the **Results page**.

In your own code, what route name did you use when defining your Results page?

I just titled the route Results but I never coded anything

That's totally fine! You can build the Results page now. You don't need to have coded it before connecting to it.

In Drafter, a page needs two things:

- A **route decorator**, which defines the URL.
- A **function**, which returns the page the player sees.

For example, here's a small unrelated example of a page route:

```
@route("/summary")
def summary(state: State) -> Page:
    return Page(state, ["Your summary"])
```

For your assignment, you'll create a route for Results and then change the final `return Page(...)` in `test_bridge` so a stable bridge goes there.

**One question:** Do you want the Results page to display the remaining budget and whether the bridge passed its stability check?

Ok here's a thought: Because I was going to plan out the message by myself but I had a better idea since my 3rd if statement doesn't make much sense now, I was thinking. What if I make the message for results say 2 different things? Because I know you said I don't need to make another if statement since the goal is for the bridge to pass regardless but I figure, I make it anyway so that way the result would immediately be: "10/10 job. Bridge is completely stable and there is no more traffic jams in the city. Great job, civil engineer!" But if they failed at first and then redid it, the end message would say: 7/10 job. Bridge did not pass at first but then passed the other time. Good job, civil engineer

I like that idea! You're giving the player feedback based on **how they reached success**, not just whether the bridge eventually passed.

The important distinction is that your program needs to remember whether the bridge failed its stability test before it passed.

Think of it as two possible histories:

- **Passed on the first test:** 10/10 message.
- **Failed first, redesigned, then passed:** 7/10 message.

You don't need a third `if` statement just to display those messages. You need a way to track that earlier failure, then use that information on the Results page to decide which message to show.

In the textbook's **1B2 Tracing Reading**, variables retain values that can be read later. That's the concept you can use here.

One question to think through: **What information could you store in `State` to remember that the bridge failed its first test?**

well no see heres the thing- I need a 3rd if statement because that's what the assignment asks for- a minimum of 3

Ohh, that changes things! If the assignment requires **at least 3 `if` statements**, then yes, you need a third one. My earlier suggestion was based on the program logic alone, not the assignment requirement.

Your two different Results messages are a reasonable purpose for that third `if`.

You could have the third condition check whether the bridge had failed a previous stability test. That gives the condition a meaningful job rather than adding an `if` statement just to meet the requirement.

The key concept from **1B2: Tracing Reading** is that you can store information in a variable and check its value later.

So your design has three decisions:

1. Is the player over budget?

2. Is the bridge unstable?
3. Did the bridge fail an earlier test before eventually passing?

That third decision fits your idea nicely.

yes I like it too alright lets get started! how do I code this

Let's build it one piece at a time!

**Step 1: Track whether the bridge has failed before.**

Your program needs to remember the first stability failure, even after the player returns to redesign the bridge.

You can do that with a Boolean variable in your `State` dataclass. A Boolean can hold either `True` or `False`.

For example, in an unrelated context:

has_failed = False

Initially, the player hasn't failed, so the value is `False`.

**Your turn:** In your `State` dataclass, what variable name would you choose to represent whether the bridge has failed a previous test? Think of a name that clearly communicates what `True` means.

This connects to **1B1: Variables Reading**—a good variable name should make its purpose clear.

previously_failed_bridge

That name communicates the idea! `previously_failed_bridge` is a Boolean variable.

**Step 2: Add it to your `State` dataclass.**

What initial value should it have when a player starts a brand-new game: `True` or `False`?

Remember, at the beginning, the bridge hasn't failed any tests yet.

Ok so false then

Exactly! Since the bridge hasn't failed yet, its initial value should be `False`.

Add `previously_failed_bridge` as a Boolean field in your `State` dataclass and initialize it to `False` where you create your starting `State`.

**Step 3:** Now think about your existing stability check in `test_bridge`.

When the bridge is unstable, where should `previously_failed_bridge` change from `False` to `True`—before or after the program returns the player to redesign the bridge?

Check the order of statements: Python executes them from top to bottom (**1B3: Modules Reading**).

Before?

Yes! Think about what happens when Python reaches the `return` statement: the function exits immediately.

So you need to update `previously_failed_bridge` **before** the `return` sends the player back to redesign.

That way, the value stays `True` when the player comes back for another test.

Go ahead and add that assignment inside your unstable-bridge `if` block, just before its `return`.

ok so still- how would I code that out

Since this is part of your assignment, I'll help you place it without writing the assignment code for you.

Inside your existing `if` statement that checks whether the bridge is unstable, you already have a `return Page(...)`.

You need to add **one assignment statement immediately before that return**.

The structure looks like this in an unrelated example:

```
if oven.temperature < 350:
    cake_needs_more_heat = True
    return "Keep baking"
```

For your code:

- The variable being updated is `previously_failed_bridge`.
- The value you're assigning is `True`.
- The assignment belongs inside the unstable-bridge block.

**Your turn:** Try writing that one assignment statement.

im still confused how am I supposed to use that example and convert it into what I want T_T

Aww, don't worry! T_T I think I made that more confusing than it needed to be. Let's use your actual variable names.

You already have this condition in your code:

```
if state.current_bridge.stability == False:
```

Inside that block, you want to change the value of `previously_failed_bridge` to `True`.

Remember the assignment pattern:

```
variable_name = new_value
So, using your own variable name:
state.previously_failed_bridge = True
```

That's the one statement you need to add **inside your stability `if` block, immediately before the `return Page(...)`**.

Why? Because the program needs to remember the failure before sending the player back to redesign.

This connects to the textbook's **1B2: Tracing Reading**: an assignment statement updates the value stored in a variable.



ok i have this

You're getting there! I can see what you're trying to do. There are **two things to fix or think about** in this section.

**1. Your stability check needs to remember the failure**

Look at this part:

```
if state.current_bridge.stability == False:
    return Page(...)
```

Right now, the function returns immediately when the bridge is unstable. But where does `previously_failed_bridge` get changed to `True`?

Remember, we discussed putting that assignment **inside this block, before the** `return`.

**2. Your third `if` statement**

You wrote:

```
if state.previously_failed_bridge = True:
```

There's a Python syntax issue here:

- `=` assigns a value to a variable.
- `==` compares two values.

An `if` condition needs an expression that evaluates to a Boolean.

**Your turn:** First, can you explain why we need to change `previously_failed_bridge` before the unstable-bridge `return`? Once you've got that, we'll work on the third `if` and your two Results messages.

This connects to **1B2: Tracing Reading**, which explains how assignment statements update variable values.

im not explaining that lets get straight to business

Got it. Let's focus on the code.

First, make these two changes:

**1. Inside your unstable-bridge `if` block**, put this assignment before the `return`:

```
state.previously_failed_bridge = True
```

**2. For your third `if` statement**, the syntax issue is that you're using `=` instead of `==`. An `if` condition compares values, so it needs the equality operator.

Your condition should check whether:

state.previously_failed_bridge

is `True`.

Put that third `if` on the Results page, where you'll choose between the 10/10 and 7/10 messages.

That's the structure. The textbook's **1A7 Logic Reading** covers `==` for comparisons, while **1B2 Tracing Reading** covers updating variables with `=`.

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost +
total_aluminum_cost
    state.remaining_budget = state.starting_budget - total_material_cost
    if state.remaining_budget < 0:
        return Page (state, [("You have went over budget. Please go back and find a better solution"), Button("Return to
Materials", "choose_materials")])
    if state.current_bridge.stability == False:
            if state.previously_failed_bridge = True:
                return Page(state, [
                    "Your bridge is unstable! You must redesign it.", Button("Restart Bridge", "bridge_building")
                    ]) better now?
```

You're closer, but there are **two things to fix** in that section:

1. `=` **is assignment, not comparison.** In an `if` condition, use `==` to compare the value. Your textbook's *1A7 Logic Reading* covers comparison operators.
2. **The nested** `if` **is checking the wrong thing.** You want to record that the bridge failed, not check whether it had already failed. Set `state.previously_failed_bridge` to `True` inside the unstable-bridge block, before the `return`.

The structure should be:

```
if state.current_bridge.stability == False:
    state.previously_failed_bridge = True
    return Page(state, [
        "Your bridge is unstable! You must redesign it.",
        Button("Restart Bridge", "bridge_building")
    ])
```

Also, your budget check comes before the stability check, which is a sensible order. *1B2 Tracing Reading* explains why the assignment must happen before `return`: once Python executes `return`, the function exits.

Your third `if` for the 7/10 versus 10/10 message belongs on the Results page, after the bridge passes.

Yeah I wanted to ask about that too- where do I put my second message on the results page

Put the second message **inside your `results` route**, as the alternative to the first message.

The structure is an `if` / `else`:

```
if state.previously_failed_bridge == True:
    # Display your 7/10 message here
else:
    # Display your 10/10 message here
```

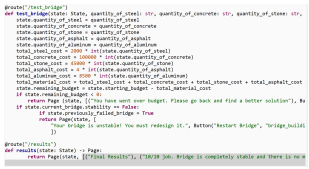
So:

- `if` handles the bridge that failed once, then passed.
- `else` handles the bridge that passed on the first test.

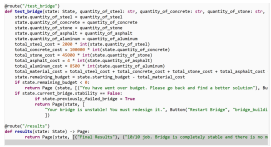
Both messages should be displayed using the `Page` returned by your Results route. You don't need to put either message back in `test_bridge`.

Your `test_bridge` route's stable-bridge path will also need to send the player to `"results"`.

*Textbook connection: 1A7 Logic Reading covers Boolean comparisons, and 1B3 Modules Reading explains how Python executes statements in order.*


hold on- i'll show you what i have entirely

No problem—send the full code when you're ready. I'll wait and review it all together before suggesting changes.


I can see the `test_bridge` section and the beginning of `results`, but the code is cut off on the right and the Results page isn't fully visible.

Send the remaining portion, especially the complete `results` function. I'll review everything together before suggesting changes.

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
```

```
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost +
total_aluminum_cost
    state.remaining_budget = state.starting_budget - total_material_cost
    if state.remaining_budget < 0:
        return Page (state, [("You have went over budget. Please go back and find a better solution"), Button("Return to
Materials", "choose_materials")])
    if state.current_bridge.stability == False:
            if state.previously_failed_bridge = True
            return Page(state, [
                "Your bridge is unstable! You must redesign it.", Button("Restart Bridge", "bridge_building")
                ])

@route("/results")
def results(state: State) -> Page:
        return Page(state, [("Final Results"), ("10/10 job. Bridge is completely stable and there is no more traffic jams in
the city. Great job, civil engineer!")
```

I can see what you're trying to do. You're close! There are **three things to address**.

**1. Fix the unstable-bridge section**

This line has a syntax error:

```
if state.previously_failed_bridge = True
```

You don't need another `if` here. You want to **record** that the bridge failed, so use an assignment statement directly inside the stability check, before its `return`.

That follows the assignment-statement concept in *1B2 Tracing Reading*.

**2. Put both messages in the Results route**

Your Results route currently only displays the 10/10 message.

Inside `results`, put your conditional before the `return Page(...)` that currently displays that message:

```
if state.previously_failed_bridge == True:
    # Return a Page displaying your 7/10 message
else:
    # Return a Page displaying your 10/10 message
```

The `if` branch handles a bridge that failed once and then passed. The `else` branch handles a bridge that passed on the first test.

This uses the Boolean comparison concepts from *1A7 Logic Reading*.

**3. Make sure the stable bridge reaches Results**

Your `test_bridge` function currently has no return for the case where the bridge is stable and within budget. Without one, Python will finish the function without displaying the Results page.

You'll need a return that navigates to your `"results"` route.

**One important detail:** Make sure `previously_failed_bridge` is initialized to `False` in your State setup, so a first-try success gets the 10/10 message.

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost +
total_aluminum_cost
    state.remaining_budget = state.starting_budget - total_material_cost
    if state.remaining_budget < 0:
        return Page (state, [("You have went over budget. Please go back and find a better solution"), Button("Return to
Materials", "choose_materials")])
    if state.current_bridge.stability == False:
        state.previously_failed_bridge = True
        return Page(state, [
            "Your bridge is unstable! You must redesign it.", Button("Restart Bridge", "bridge_building")
            ])

@route("/results")
def results(state: State) -> Page:
    if state.previously_failed_bridge == True:
        return Page(state, [("Final Results"), ("7/10 job. Bridge did not pass at first but then passed the other time. Good
job, civil engineer")
        else:
            return Page(state, [("Final Results"), ("10/10 job. Bridge is completely stable and there is no more traffic jams
in the city. Great job, civil engineer!")
```

ok this is it so far

You're making good progress. Your logic is in place, but there are **two remaining issues** to address.

**1. Fix the Results route syntax**

Your `if` branch's `return Page(...)` is missing its closing `])` before `else`.

Also, `else` must line up vertically with `if`, not with `return`.

The structure should look like this:

```
if condition:
    return Page(state, [
        "Final Results",
        "Your 7/10 message"
    ])
else:
    return Page(state, [
        "Final Results",
        "Your 10/10 message"
    ])
```

This is just a structural example; keep your own messages and condition.

**2. Add the stable-bridge path in `test_bridge`**

Right now, `test_bridge` handles:

- Over budget.
- Unstable bridge.

But it doesn't yet handle a bridge that is **within budget and stable**. At the end of that function, you need a return that takes the user to the `"results"` route.

**One textbook connection**

In *1B3 Modules Reading*, Python executes statements sequentially. Each `return` exits the function, so the stable-bridge return needs to come after your two existing `if` checks.

Fix those two things, and your route logic will be much closer to complete.

Ok wait so where would I put the results button

Put the Results button **at the end of your `test_bridge` function**, after the unstable-bridge `if` block.

It should be the final return, so it only runs if:

- The player is not over budget.
- The bridge is stable.

For example, the navigation pattern in Drafter is:

```
return Page(state, [
    "Bridge passed!",
    Button("See Results", "results")
])
```

Your Results route is already named `"results"`, so that button destination matches it.

**Important:** Keep this return outside the unstable-bridge `if` block, aligned with that `if`. That way, an unstable bridge won't reach the Results button.

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost +
total_aluminum_cost
    state.remaining_budget = state.starting_budget - total_material_cost
    if state.remaining_budget < 0:
        return Page (state, [("You have went over budget. Please go back and find a better solution"), Button("Return to
Materials", "choose_materials")])
    if state.current_bridge.stability == False:
        state.previously_failed_bridge = True
        return Page(state, [
            "Your bridge is unstable! You must redesign it.", Button("Restart Bridge", "bridge_building")
            ])
        return Page(state, [
            "Bridge has passed! Click to see results",
            Button("See results", "results")
            ]) better?
```

Almost! Your Results button is in the right function, but there's **one indentation problem**.

Right now, the Results `return Page(...)` is:

- Inside the unstable-bridge `if` block.
- After another `return`, which means Python can never reach it.

**Move the Results return back one indentation level.** It should line up with your two `if` statements, like this:

```
if state.current_bridge.stability == False:
    state.previously_failed_bridge = True
    return Page(state, [
        "Your bridge is unstable! You must redesign it.",
        Button("Restart Bridge", "bridge_building")
    ])

return Page(state, [
    "Bridge has passed! Click to see results",
    Button("See Results", "results")
])
```

Now the Results button only appears if the player is within budget and the bridge is stable.

This follows *1B3 Modules Reading*: Python executes statements in order, and a `return` exits the function immediately.

```
@route("/test_bridge")
def test_bridge(state: State, quantity_of_steel: str, quantity_of_concrete: str, quantity_of_stone: str,
quantity_of_asphalt: str, quantity_of_aluminum: str) -> Page:
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    total_steel_cost = 2000 * int(state.quantity_of_steel)
    total_concrete_cost = 100000 * int(state.quantity_of_concrete)
    total_stone_cost = 45000 * int(state.quantity_of_stone)
    total_asphalt_cost = 4 * int(state.quantity_of_asphalt)
    total_aluminum_cost = 8500 * int(state.quantity_of_aluminum)
    total_material_cost = total_steel_cost + total_concrete_cost + total_stone_cost + total_asphalt_cost +
total_aluminum_cost
    state.remaining_budget = state.starting_budget - total_material_cost
    if state.remaining_budget < 0:
        return Page (state, [("You have went over budget. Please go back and find a better solution"), Button("Return to
Materials", "choose_materials")])
    if state.current_bridge.stability == False:
        state.previously_failed_bridge = True
        return Page(state, [
            "Your bridge is unstable! You must redesign it.",
            Button("Restart Bridge", "bridge_building")
        ])
        return Page(state, [
            "Bridge has passed! Click to see results",
            Button("See results", "results")
    ]) so like this??
```

Almost! Your indentation is still one level too far in for the final `return Page(...)`.

The Results return needs to line up with the two `if` statements, **not** sit inside the stability `if`.

The structure should be:

```
if state.current_bridge.stability == False:
    state.previously_failed_bridge = True
    return Page(state, [
        "Your bridge is unstable!",
        Button("Restart Bridge", "bridge_building")
    ])

return Page(state, [
    "Bridge has passed! Click to see results",
    Button("See results", "results")
])
```

Compare the indentation carefully:

- The unstable-bridge return is inside the `if`.
- The Results return is outside the `if`.

That's the key distinction. *1B3 Modules Reading* explains how Python uses indentation to determine which statements belong to a block.

How though???

I see the issue now! **Your indentation is actually correct in the screenshot.** I was describing it unclearly.

Look at the two `return` lines:

- The unstable-bridge `return` should be indented **inside** the `if`.
- The Results `return` should start at the **same indentation as the** `if`.

In your screenshot, the Results `return` starts with 4 spaces, and the `if` also starts with 4 spaces. That's what you want!

One more thing: your screenshot shows the unstable-bridge `return` aligned with `state.previously_failed_bridge`, but it needs to be indented one level farther so it belongs inside the `if`.

*Textbook connection: 1B3 Modules Reading explains that indentation determines how Python groups statements into blocks.*

wait so which return do you want me to indent

Indent the **first** `return` (the one displaying “Your bridge is unstable!”) one level farther in.

- The unstable-bridge `return` goes **inside** the `if`.
- The Results `return` stays **outside** the `if`, lined up with `if state.current_bridge.stability == False:`.

So, don't indent the Results `return` any further. It's already at the right level.

*Textbook: 1B3 Modules Reading — indentation determines which statements belong to an `if` block.*

```
if state.current_bridge.stability == False:
        state.previously_failed_bridge = True
        return Page(state, [
            "Your bridge is unstable! You must redesign it.", Button("Restart Bridge", "bridge_building")
            ])
    return Page(state, [
            "Bridge has passed! Click to see results",
            Button("See results", "results")
            ]) is this good or not
```

Yes! **Your indentation is now correct.** The unstable-bridge `return` is inside the `if`, and the Results `return` is outside it.

One small thing to remember: this final `return` displays the Results button whenever the bridge is stable and the budget hasn't been exceeded.

Your next step is to run it and test both cases:

- Unstable bridge → restart message.
- Stable bridge → See Results button.

*Textbook connection: 1B3 Modules Reading explains how Python uses indentation to group statements and execute them in order.*

im not running it yet now here comes the for loop: for criteria in bridge.qualities:

#loop iterates through list of Criteria instances stored in class Bridge qualities attribute; allows program to work on budget, time duration and number of construction workers ; i need a minimum of 1 so theres that

Good! This can satisfy your minimum of one `for` loop.

Your loop header has three important parts:

- `criteria` — the iteration variable. It represents one `Criteria` object at a time.
- `in` — connects the iteration variable to the collection.
- `bridge.qualities` — the list being iterated over.

**One thing to check:** In your earlier State setup, your Bridge object was stored as `state.current_bridge`. Have you defined a separate variable named `bridge`, or should this loop refer to that State attribute?

Also, your comment explains the purpose well. The *6A1 For Loops Reading* in your textbook explains that each list element gets assigned to the iteration variable, one at a time.

i think the second one, i don't wanna do too much

That's simpler. Use the existing `state.current_bridge` instead of creating another variable.

Your loop header would refer to:

for criteria in state.current_bridge.qualities:

Keep it at the same indentation level as the other statements inside the function.

That way, you're iterating through the `Criteria` objects already stored in your current Bridge. No extra variable or setup needed.

*Textbook connection: 1B1 Variables Reading explains accessing values through variable names and attributes.*

ok wonderful where and how in my code do i set that up

Put the `for` loop **inside your** `test_bridge` **function**, after your budget calculation and before the final Results `return`.

Specifically, place it after this section:

state.remaining_budget = state.starting_budget - total_material_cost
Then start the loop at the same indentation level as your `if` statements:
for criteria in state.current_bridge.qualities:

**Important:** The loop header only starts the loop. The statements you want repeated must be indented underneath it.

Before adding those statements, let's establish what the loop should inspect: your `Criteria` class has `budget`, `time_duration`, and `construction_workers`.

*Textbook connection: 6A1 For Loops Reading explains how a `for` loop visits each item in a collection one at a time.*

ok yeah see i didn't really know where I was going with this so I don't know what to do now

That's completely okay. We don't need to force a `for` loop into your program just to meet the requirement. Let's first give it a purpose.

Looking at your `Criteria` class, each object stores three things:

- `budget`
- `time_duration`
- `construction_workers`

And your `Bridge` class stores a list of those `Criteria` objects in `qualities`.

So a loop could be useful if your simulator needs to **check whether a bridge meets each construction criterion**.

For example, imagine a bridge has to meet the budget, time, and worker requirements before it can pass testing. The loop could visit each `Criteria` object and let your program examine its values.

The textbook's **6A1 For Loops Reading** explains that a `for` loop processes items in a collection one at a time.

**Let's start with the purpose:** Does your bridge need to pass all three criteria—budget, construction time, and number of workers—before it can be considered successful?

I'm too tired for it to have a purpose, lets just get the function done and over with so I can move along

I hear you. Let's keep this simple and get you moving.

We can use the `for` loop to examine the `Criteria` objects already stored in your bridge. You don't need to invent a new feature or add more data.

In your `test_bridge` function, find this line:

state.remaining_budget = state.starting_budget - total_material_cost

Put the loop **immediately below that line, before your first `if` statement**.

The loop header is:

for criteria in state.current_bridge.qualities:

One important thing: Python requires an indented statement inside the loop, so don't leave the loop empty.

Your `criteria` variable represents one `Criteria` object at a time. Its attributes are accessible with dot notation, such as `criteria.budget`.

This follows the textbook's **6A1 For Loops Reading**: a loop processes items in a collection one at a time.

Let's get that structure in place first; we can keep the rest of your function intact.

ok yeah typed the for loop out now what

Nice. Now we need to give the loop a statement to execute.

Since `criteria` represents a `Criteria` object, it can access the object's three attributes:

- `criteria.budget`
- `criteria.time_duration`
- `criteria.construction_workers`

For now, let's make sure you understand what the loop is visiting.

**Your `state.current_bridge.qualities` list currently contains one `Criteria` object.** So how many times do you expect the loop body to run?

The textbook's **6A1 For Loops Reading** describes how the loop variable takes on each item in the collection, one at a time.

Once

Exactly—one iteration, because your list currently contains one `Criteria` object.

Now let's make that iteration useful without adding a complicated feature.

Inside the loop, we can start by examining the budget stored in that object. In Python, `criteria.budget` accesses its budget attribute.

**What do you think `criteria.budget` evaluates to with your current initialization?**

budget in criteria

Yes! More precisely, it evaluates to **600000**, because your `Criteria` object was initialized with `budget = 600000`.

That means `criteria.budget` accesses the integer stored in the object's `budget` attribute.

Now you've seen how the loop variable lets you access data inside each object. That's the key idea behind this loop. The textbook's **1B1 Variables Reading** explains how names let us access stored values.

ok lets just please put it in the for loop already
Put this indented directly under your `for` loop header:
required_budget = criteria.budget
So the loop looks like:

```
for criteria in state.current_bridge.qualities:
    required_budget = criteria.budget
```

That gives the loop a real statement to execute. Since your list currently contains one `Criteria` object, `required_budget` will be assigned `600000` once.

This uses **1B1 Variables Reading**: a variable name lets you refer to a value, and **1B2 Tracing Reading**: Python evaluates the right side before assigning it to the left side.

ok is my for loop done is that it
Yes! **Your `for` loop is syntactically complete**:
for criteria in state.current_bridge.qualities:
    required_budget = criteria.budget

It has:

- A `for` loop header.
- A collection to iterate through: `state.current_bridge.qualities`.
- A loop variable: `criteria`.
- An indented statement inside the loop.

Since your list currently has one `Criteria` object, the loop runs once and assigns `600000` to `required_budget`.

One distinction: the loop is complete, but `required_budget` is not currently used elsewhere in your function. That's okay if your assignment only requires demonstrating a `for` loop. **1B3 Modules Reading** also covers how Python executes statements sequentially and uses indentation to group a loop body.

oh my god i'm finally in the homestretch of this FINALLY OK now help me figure out where to put the 2 other inputs:
CheckBox: Players can decide which optional bridge features are actually needed - selecting or unselecting material options could potentially alter the construction cost

TextArea: Allow players to give an explanation for their engineering decisions & why they selected the materials they did

YOU'RE ALMOST THERE! 🎉 Let's put these in the right places without disrupting the work you've already done.

Both inputs belong on your `choose_materials` **route**, because that's where players make their material and design decisions.

**1. CheckBox — optional bridge features**

Put the CheckBox input in `choose_materials`, **after the material quantity inputs and before the Continue button**.

You already have `state.optional_features`, so this input can store which features the player selects.

A question to consider: does your assignment expect players to select multiple optional features, or just one? That affects how you configure the CheckBox input.

**2. TextArea — engineering explanation**

Put the TextArea on that same `choose_materials` page, **after the CheckBox section and before Continue**.

It should let players explain:

- Why they selected those materials.
- Why they chose those quantities.
- Why they selected or skipped optional features.

You'll need a State variable to store the explanation, just as you have State variables for the material quantities.

**Your page layout**

**Choose Your Materials**

- Steel quantity
- Concrete quantity
- Stone quantity
- Asphalt quantity
- Aluminum quantity
  **Optional Bridge Features**
- CheckBox options
  **Engineering Decisions**
- TextArea for explanation
  **Continue** → Test Bridge

One important thing: your `test_bridge` route will also need to receive and store the new input values, just like it does for the five material quantities.

This connects to **1A3 Console I/O Reading**: your page gathers input, your route processes it, and your results page can present the outcome.

ok so would it look something like this: def index(state: State) -> Page:
    return Page(state, [
        Header("Babbage's Day"),
        "Walked: " + str(state.walked) + "\n",
        "Fed: " + str(state.fed) + "\n",
        CheckBox("walked_today", state.walked),
        " Took Babbage for a walk\n",
        CheckBox("fed_today", state.fed),
        " Fed Babbage\n",
        Button("Save", "save")
    ])

Yes! That's exactly the kind of structure you want. You're using `CheckBox` inside the `Page` components list, just like your example.

For your bridge project:

- **CheckBox:** Goes inside the `Page` list on `choose_materials`, after the material quantity inputs.
- **TextArea:** Also goes inside that same `Page` list, after the optional features and before the Continue button.

One important distinction: your example uses `state.walked` and `state.fed` to store checkbox values. Your bridge checkboxes should connect to your optional-feature State variable.

For the TextArea, you'll need a separate State variable to store the player's explanation.

**Textbook connection:** See *1B1 Variables Reading* for how State variables hold values, and *1A3 Console I/O Reading* for collecting user input.

You're on the right track!



```
@route("/choose_materials")
def choose_materials(state: State) -> Page:
    return Page(state, [
        "Choose Your Materials" + "\n",
        "Amount of steel needed:",
        TextBox("quantity_of_steel", state.quantity_of_steel),
        "\n",
        "Amount of concrete needed:",
        TextBox("quantity_of_concrete", state.quantity_of_concrete),
        "\n",
        "Amount of stone needed:",
        TextBox("quantity_of_stone", state.quantity_of_stone),
        "\n",
        "Amount of asphalt needed:",
        TextBox("quantity_of_asphalt", state.quantity_of_asphalt),
        "\n",
        "Amount of aluminum needed:",
        TextBox("quantity_of_aluminum", state.quantity_of_aluminum),
        "\n",
        def index(state: State) -> Page:
    return Page(state, [
        Header("Babbage's Day"),
        "Walked: " + str(state.walked) + "\n",
        "Fed: " + str(state.fed) + "\n",
        CheckBox("walked_today", state.walked),
        " Took Babbage for a walk\n",
        CheckBox("fed_today", state.fed),
        " Fed Babbage\n",
        Button("Save", "save")
    ])
        Button("Continue", "test_bridge"),
    ])
```

so like this?

Almost! You're copying the right idea, but there's one important placement mistake.

**Don't put the entire** `def index(state: State)` **function inside** `choose_materials`**.** That is a separate example route. You only need to copy the `CheckBox(...)` component pattern.

In your screenshot:

1. Keep all five material TextBoxes exactly where they are.
2. Remove the pasted `def index(...)` and its entire `return Page(...)` block.
3. Add your bridge feature CheckBoxes **inside the existing** `Page` **list**, after the aluminum TextBox.
4. Add your TextArea after the CheckBoxes.
5. Keep `Button("Continue", "test_bridge")` as the final component in that list.

Your structure should be:

**Existing material inputs**

Steel, concrete, stone, asphalt, aluminum

**New optional feature inputs**

Your CheckBox components go here.

**Engineering explanation**

Your TextArea goes here.

**Continue button**

Remember: every component must be inside the same `Page` list, with matching indentation and commas.

This connects to **1B3 Modules Reading**: indentation determines which statements belong inside a function or block.

You're very close!

wait but it'd only be for the optional features- so there'd be only 3 options

Ohhh, yes! Exactly. You will have **only 3 CheckBoxes**, one for each optional bridge feature.

You do NOT need a CheckBox for each material. Your five material choices remain TextBoxes.

Your `choose_materials` page will have:

1. Five material quantity TextBoxes.

2. Three optional-feature CheckBoxes.
3. One TextArea for the engineering explanation.
4. The Continue button.

Since you already have `state.optional_features`, we can use that for the selected features.

**One thing to clarify:** What are the names of your three optional bridge features? Then we can set up those CheckBoxes.

Wait, I already have that state option? Where??
You already included it in your **State initialization** earlier! It was this part:

```
bridging_off = State(
    selected_material=None,
    remaining_budget=600000,
    starting_budget=600000,
    optional_features=[],
    current_bridge=starting_bridge,
    name_of_bridge="",
    list_of_criteria=[quantity]
)
```

See `optional_features=[]`? That's where the list starts empty.

You should also have this field declared in your `State` dataclass:

optional_features: list[str]

- The State field defines the variable.
- `optional_features=[]` initializes it as an empty list.
- Your three CheckBoxes will let players select optional features.

This connects to **1B1 Variables Reading**: variables give us names to access stored values.

So you don't need to create another optional-features variable!

oh wait so i need both types for optional_features?

Yes! You need **both**, but they serve different purposes.

**1. In your `State` dataclass:**

optional_features: list[str]

This declares the variable's type: a list of strings.

**2. In your State initialization:**

optional_features=[]

This gives the variable its starting value: an empty list.

Think of it this way:

- The dataclass declaration says what kind of data it holds.
- The initialization gives it its initial value.

You already have both from what you showed me earlier, so **you don't need to add them again!**

This relates to **1B1 Variables Reading**: variables are labels attached to values.

wait nvm I just checked I have both lol

Haha, perfect! 😂 One less thing to worry about.

You're all set on the State side. We can move on to adding the three CheckBoxes and the TextArea to `choose_materials`.

You're getting really close to the finish line!

Thank god tell me what to do

YES. Let's get these last two inputs added. We'll do one part at a time.

**Step 1: Add the three CheckBoxes**

Open your existing `choose_materials` function.

Find the end of your aluminum input:

"Amount of aluminum needed:",
TextBox("quantity_of_aluminum", state.quantity_of_aluminum),
"\n",

**Immediately after that**, add three `CheckBox` components, one for each optional feature.

The structure follows the example you showed me:

CheckBox("feature_name", False),
" Feature description\n",

Those are illustrative names and values, not your final feature choices. Use your actual feature names.

**Step 2: Add the TextArea**

Immediately after the three CheckBoxes, add a `TextArea` component for the engineering explanation.

You'll also need a new State variable to store that explanation, with a string type and an initial empty-string value.

**Step 3: Keep Continue at the bottom**

Your existing:

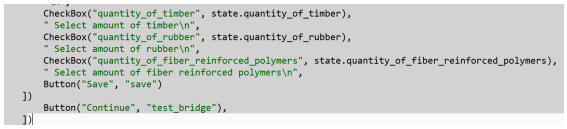
Button("Continue", "test_bridge"),

stays at the very bottom of the `Page` list.

**One important thing:** Your `test_bridge` route must receive the checkbox selections and explanation too, so those values aren't lost when the player continues.

This follows **1A3 Console I/O Reading**: collect input first, then process it. **1B2 Tracing Reading** explains how those values are stored and read.

Let's start with Step 1: what are the names of your three optional features?



ok thoughts

A couple of things stand out. **The main issue is that these are quantity fields, but you're using `CheckBox` components.**

For example:

CheckBox("quantity_of_timber", state.quantity_of_timber)

A checkbox represents a **yes/no choice** (Boolean), not an amount.

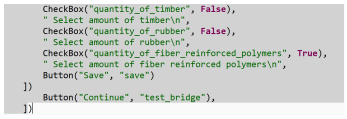
- If the player needs to enter how many tons of timber, use a `TextBox`.
- If the player only needs to choose whether timber is included, a `CheckBox` makes sense.

Also, your `quantity_of_timber`, `quantity_of_rubber`, and `quantity_of_fiber_reinforced_polymers` State variables are currently strings. A checkbox should be connected to a Boolean State variable instead.

One more thing: your `Button("Continue", "test_bridge")` is outside the inner list that ends with `])`. That may be fine depending on the surrounding code, but check that the brackets match your `Page` structure.

**My recommendation:** Keep quantity inputs as TextBoxes, and use your three CheckBoxes only for the three optional bridge features.

This connects to the textbook's **1B1 Variables Reading** (variables store values) and **1B2 Tracing Reading** (the value assigned to a variable needs to match how the program uses it).



how bout now

Much better! You're now passing `False` or `True`, which are Boolean values—the correct kind of value for a checkbox.

There are just two things to address:

1. **Rename the checkbox identifiers.** `quantity_of_timber`, for example, sounds like an amount. Use names that describe whether a feature is selected.
2. **Change the labels.** `"Select amount of timber"` still sounds like a quantity input. The label should describe the optional feature the player is choosing.

Your `False` and `True` values are fine:

- `False` = unchecked initially.
- `True` = checked initially.

One important distinction: these checkboxes should represent your **three optional bridge features**, not your three additional material quantities.

See **1A7 Logic Reading** in the textbook: Boolean values represent `True` or `False`.

You're on the right track.

wait so what do you want me to change for #1

I mean change the **first argument inside each** `CheckBox()` so it describes an optional feature instead of a material quantity.

For example, change:

CheckBox("quantity_of_timber", False)
to something like:
CheckBox("has_lighting", False)

Do the same for the other two, using names that describe your actual optional features.

Keep `False` or `True` as the second argument. **You don't need to change those.**

The identifier should answer: “Is this feature selected?” rather than “How much material is needed?”

ohhhh wait so it's something entirely different then what I have???

Yes—but don't worry, you haven't messed anything up! We were talking about two different things.

You have **two separate categories**:

1. **Materials:** Steel, concrete, stone, asphalt, aluminum, timber, rubber, and fiber-reinforced polymers. These need quantity inputs (`TextBox`) because players enter how much they want.
2. **Optional bridge features:** These are extra design choices that players turn on or off (`CheckBox`).

So your timber, rubber, and fiber-reinforced polymer inputs should remain quantity inputs. The three CheckBoxes should be for three additional features.

**You don't need to delete your material inputs.** We just need to keep the two categories separate.

This follows the textbook's **1A3 Console I/O Reading**: inputs collect information that the program processes later.

```
CheckBox("uses_timber", False),
    " Select amount of timber\n",
    CheckBox("uses_rubber", False),
    " Select amount of rubber\n",
    CheckBox("uses_fiber_reinforced_polymers", True),
    " Select amount of fiber reinforced polymers\n",
    Button("Save", "save")
```

]) now

Yes! The checkbox identifiers now make sense. You're using names like `uses_timber` to represent whether a material is selected. 👍

**One thing still needs changing:** your labels say `"Select amount of timber"`, but a checkbox doesn't select an amount. It selects whether something is included.

For example, if timber is an optional feature, the label could be `"Include timber in bridge\n"`.

Also, make sure these are actually your **three optional features**, rather than three additional materials.

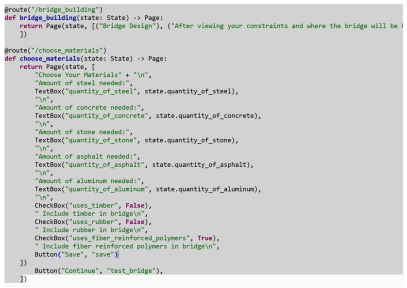
Your `False`, `False`, and `True` defaults are valid Boolean values.

**Textbook connection:** *1B1 Variables Reading* explains how variable names help communicate what information a variable represents. Your `uses_timber` name does that much better than `quantity_of_timber`.

ok i just changed both still getting the grey stuff

What do you mean by the grey stuff—is the **code appearing greyed out in your editor**, or are the **checkboxes appearing grey in your running app**?

If you can send a screenshot of what you're seeing, I can help pinpoint it.



here this is what im talking about

The grey highlighting is covering your code because **your brackets are misplaced**.

Look at the bottom of `choose_materials`.

Right now, you have:

```
Button("Save", "save")
])
Button("Continue", "test_bridge"),
])
```

The first `])` already closes your `Page`, so the Continue button ends up outside it.

**Here's what to change:**

1. Remove the `Button("Save", "save")` line for now.
2. Remove the first `])` immediately below it.
3. Keep `Button("Continue", "test_bridge")` inside the Page list.

4. Keep the final `])` to close the Page.

That addresses the bracket problem. The grey background itself may simply be your editor's highlighting, so check whether the code runs after fixing the structure.

This relates to the textbook's **1B3 Modules Reading**: Python uses indentation and syntax to organize statements.

ok yeah it all went away

Great! That means the misplaced brackets were causing the issue.

Your `choose_materials` page is now structured properly, so we can move on to getting the checkbox selections saved when the player clicks **Continue**.

Nice progress!

ok now for TextArea: how and where do i code that

You'll put the `TextArea` **inside your** `choose_materials` **function**, directly after your three CheckBoxes and **before the Continue button**.

The purpose is to let players explain why they chose their bridge materials.

**Step 1: Add a State variable**

In your `State` dataclass, add a string field for the explanation. For example:

engineering_explanation: str

Then initialize it to `""` wherever you create your starting `State`.

**Step 2: Add the TextArea**

Find your last checkbox:

CheckBox("uses_fiber_reinforced_polymers", True),
" Include fiber reinforced polymers in bridge\n",
**Directly underneath it**, add a descriptive label and a `TextArea`. Its structure is similar to your existing `TextBox`:
TextArea("engineering_explanation", state.engineering_explanation),

Keep the **Continue** button underneath the TextArea, inside the same `Page` list.

**Step 3: Save the player's explanation**

Your `test_bridge` route will also need to receive the `engineering_explanation` input and store it in `state`, just like it currently does for your material quantities.

The textbook's **1B1 Variables Reading** explains why we need a named variable to store the player's text.

**For now, focus on Steps 1 and 2.** Once those are in place, we can connect the TextArea to `test_bridge`.

ok so how do i connect that

You connect the `TextArea` to `test_bridge` the same way you connected your material TextBoxes.

There are **two places to update**.

**1. In your `test_bridge` function header**

You currently have parameters like:

quantity_of_steel: str
Add another parameter for the explanation:
engineering_explanation: str

Put it **after** `quantity_of_aluminum: str`, separated by a comma.

**2. Inside `test_bridge`**

You already have assignments like:

state.quantity_of_steel = quantity_of_steel

Immediately after your existing material assignments, add the corresponding assignment for the engineering explanation.

Remember the pattern:

**State field = incoming parameter**

For example, the left side should be `state.engineering_explanation`.

**Why this works**

When the player clicks **Continue**, Drafter sends the TextArea's contents to the route using its input name, `"engineering_explanation"`.

The route receives that text, and you store it in `state` so it remains available later.

This follows the textbook's **1B2 Tracing Reading**: an assignment evaluates the right-hand side and stores that value in the variable on the left.

**One quick check:** What should go on the right-hand side of `state.engineering_explanation =`?

```
state.engineering_explanation = engineering_explanation
```

so like that?
Yes! That's exactly the assignment you need:
state.engineering_explanation = engineering_explanation

The right side is the text received from the `TextArea`, and the left side stores it in your `State`.

Make sure it's **inside** `test_bridge`, directly below your other State assignments.

This follows the textbook's **1B2 Tracing Reading**: Python evaluates the right side of `=` and assigns that value to the variable on the left.

One final check: make sure `engineering_explanation: str` is also in your `test_bridge` function's parameters.

yep it is now what

Great! Your `TextArea` is now connected to `test_bridge`.

**Next, let's connect your three CheckBoxes** so the player's selections are saved too.

**Step 1: Update `test_bridge`**

Just like `engineering_explanation`, each checkbox needs a matching parameter in the `test_bridge` function header.

The difference is the **type**:

- `TextArea` → `str` (text)
- `CheckBox` → `bool` (`True` or `False`)

For example, one parameter would be:

uses_timber: bool

Add corresponding parameters for rubber and fiber-reinforced polymers.

**Step 2: Store the selections**

You already have `state.optional_features`, which is a list of strings. Eventually, you'll use the checkbox values to determine which feature names belong in that list.

For now, **add the three Boolean parameters first**.

This connects to the textbook's **1A7 Logic Reading**, where Boolean values represent `True` and `False`.

Once you've added those parameters, we can work on storing the selected features.

ok where do i do that

Inside your `test_bridge` **function**, at the very top where you list the parameters.

**Step 1: Find this line**
def test_bridge(state: State, quantity_of_steel: str, …

Go to the **end of that function header**, where you added `engineering_explanation: str`.

**Step 2: Add the checkbox parameters there**

After `engineering_explanation: str`, add a comma and these three parameters:

- `uses_timber: bool`
- `uses_rubber: bool`
- `uses_fiber_reinforced_polymers: bool`

Keep them **inside the parentheses**, before `) -> Page:`.

Don't put them inside the `return Page(...)` list.

That's all for this step! The textbook's **1A7 Logic Reading** explains why these use `bool` rather than `str`: checkboxes represent `True` or `False`.

ok so im done with all 3 user inputs?

**Almost!** 🎉 You've added all three required types of user input:

- **TextBox** — players enter material quantities. ✅
- **CheckBox** — players select optional materials. ✅
- **TextArea** — players explain their engineering decisions. ✅

The inputs are in place, but there's **one remaining step**: saving the three checkbox selections.

You've added their Boolean parameters to `test_bridge`, but you haven't yet used those values to update `state.optional_features`.

So **yes, you've added all three input types!** We just need to finish saving the checkbox choices.

This relates to the textbook's **1B2 Tracing Reading**: receiving a value and storing it for later use are separate steps.

Ok lets do that

Perfect! Let's save your three checkbox selections in `state.optional_features`.

**Step 1: Find the right spot**

Inside `test_bridge`, go directly below:

state.engineering_explanation = engineering_explanation

**Step 2: Start with an empty list**

Add:

state.optional_features = []

This ensures the list contains only the features selected during the current attempt.

**Step 3: Check each selection**

Remember, your checkbox parameters are Boolean values:

- `uses_timber`
- `uses_rubber`

- `uses_fiber_reinforced_polymers`

For each one, you'll need an `if` statement that checks whether it's `True`. When it is, add that material's name to `state.optional_features`.

Here's an unrelated example showing the pattern:

```
toppings = []
wants_sprinkles = True

if wants_sprinkles:
    toppings.append("sprinkles")
```

The textbook's **1A7 Logic Reading** explains Boolean conditions, while **1B2 Tracing Reading** explains how variables change during execution.

**Your turn:** For your first checkbox, what would the `if` condition be to check whether the player selected timber?

if they did it'd evaluate to true

Exactly! When the player checks the timber box, `uses_timber` evaluates to `True`.

In Python, an `if` statement runs its indented body when its condition is `True`.

So your next step is to write an `if` statement using `uses_timber` **as the condition**.

Then, inside that `if` block, you'll add `"timber"` to `state.optional_features` using `.append()`.

This follows the textbook's **1A7 Logic Reading** on Boolean values.

Can you try writing just that first `if` statement?

sure can, where do i put it tho
Put it **inside your** `test_bridge` **function**, directly below:
state.optional_features = []

Your order should be:

1. Save `state.engineering_explanation`.
2. Reset `state.optional_features` to an empty list.
3. Write your `if` statement for timber.
4. Add similar checks for rubber and fiber-reinforced polymers.
5. Continue with your existing material cost calculations.

Make sure the `if` statement is indented at the **same level** as `state.optional_features = []`. Its body will be indented one level further.

This follows the textbook's **1B3 Modules Reading** on indentation and execution order.

```
def test_bridge(state: State, quantity_of_steel: str, quanti
    state.quantity_of_steel = quantity_of_steel
    state.quantity_of_concrete = quantity_of_concrete
    state.quantity_of_stone = quantity_of_stone
    state.quantity_of_asphalt = quantity_of_asphalt
    state.quantity_of_aluminum = quantity_of_aluminum
    state.engineering_explanation = engineering_explanation
    state.optional_features = []
    uses_timber = True:
        if uses_timber:
            state.optional_features.append(timber)
```
good?

Almost! You're very close. There are **three small issues** in your screenshot.

1. **Remove** `uses_timber = True:`
You don't need that line. The checkbox already supplies the value of `uses_timber`.

2. **Fix the indentation.**
Your `if uses_timber:` should line up with `state.optional_features = []`. The `.append()` statement should be indented underneath the `if`.

3. **Put quotation marks around `timber`.**
In `.append(timber)`, Python treats `timber` as a variable. You want to store the material's name as a string: `"timber"`.

The textbook's **1A7 Logic Reading** explains why `if` can evaluate a Boolean directly, and **1B3 Modules Reading** covers indentation.

**Make those three changes first.** Then you'll be ready to apply the same pattern to rubber and fiber-reinforced polymers.

```
if uses_timber:
    state.optional_features.append("timber")
if uses_rubber:
    state.optional_features.append("rubber")
if uses_fiber_reinforced_polymers:
    state.optional_features.append("fiber reinforced polymers")
```
This looks much better! Your three `if` statements are correctly structured, and your `.append()` calls use strings.

Before moving on, let's trace what happens.

Suppose the player selects **timber and fiber-reinforced polymers**, but leaves rubber unchecked.

- `uses_timber` is `True`
- `uses_rubber` is `False`
- `uses_fiber_reinforced_polymers` is `True`

**What two strings would end up in `state.optional_features`?**

This connects to the textbook's **1A7 Logic Reading** (Boolean conditions) and **1B2 Tracing Reading** (tracking variable values as statements execute).

wait so you're saying i have to change that huh
**No! You don't have to change anything in that screenshot.** Your three `if` statements look good as written.

I was only asking you to trace an example to check your understanding, not telling you to edit your code.

You can keep what you have and move on.

The textbook's **1B2 Tracing Reading** covers that kind of practice.

ok cool well am i done with everything?

**You're very close to finishing!** 🎉

Based on everything we've worked on, here's your progress:

- **7 routes** — completed.
- **TextBox inputs** — added.
- **CheckBox inputs** — added and connected.
- **TextArea input** — added and connected.
- **For loop** — written.
- **Material cost calculations** — written.
- **Budget and stability conditions** — written.

**Two things still need checking before I'd call the project finished:**

1. **Results page:** You created the title, but we haven't confirmed that the `results` route is fully implemented.
2. **Testing:** We haven't confirmed that the app runs from start to finish without errors.

The textbook's **1B2 Tracing Reading** is useful for checking how values change as the program runs.

**So you're almost done, but not quite verified yet.** I'd check your Results page next, then run the simulator once from beginning to end.

oh yeah I knew about the testing part it's just, what do we need to do left for the results page?

For your **Results page**, we only have a few things left to finish!

**1. Make sure the Results route exists**

You need a function named `results` with the decorator:

@route("/results")

Remember, your **See results** button in `test_bridge` already points to `"results"`.

**2. Display the player's results**

Inside the Results page, you could show:

- **Bridge status:** Whether the bridge passed its stability test.
- **Remaining budget:** Using `state.remaining_budget`.
- **Selected optional materials:** Using `state.optional_features`.
- **Engineering explanation:** Using `state.engineering_explanation`.

You already have these values stored in `State`, so you don't need to calculate them again.

**3. Return a Page**

Just like your other routes, the `results` function needs to return a `Page` containing the information you want displayed.

The textbook's **1B1 Variables Reading** explains how stored values can be accessed later, which is exactly what you're doing here.

**Let's start with one thing:** Have you already written the `def results(state: State) -> Page:` function, or do you only have the Results page title so far?

hold up- its this: @route("/results")
def results(state: State) -> Page:
    if state.previously_failed_bridge == True:
        return Page(state, [("Final Results"), ("7/10 job. Bridge did not pass at first but then passed the other time. Good job, civil engineer")
                            ])
        else:
            return Page(state, [("Final Results"), ("10/10 job. Bridge is completely stable and there is no more traffic jams in the city. Great job, civil engineer!")
                                ])

Oh, you already have your Results page written! That's good progress.

I see **one syntax issue** you need to fix.

**The problem: `else:` indentation**

Your `else:` is currently indented underneath the first `return Page(...)`.

In Python, `else:` **must line up with its matching** `if` **statement**, not with the `return` inside it.

So:

- Keep `if state.previously_failed_bridge == True:` where it is.
- Move `else:` left until it lines up with `if`.
- Keep the second `return Page(...)` indented underneath `else:`.

This connects to the textbook's **1B3 Modules Reading**, which explains how indentation determines which statements belong to a block.

**One more thing to verify**

Your Results page checks `state.previously_failed_bridge`. Make sure that field exists in your `State` dataclass and starts as `False`.

**Other than that, your Results page has the two outcomes you wanted:** a 7/10 message for a bridge that previously failed and a 10/10 message for one that passed without failing.

Once you fix the indentation, the next step is testing the route.

alright got it, wait but before we test it answer me this- I need a minimum of 5 pages, this is gonna make me reach 5 pages right?

**Yes! You already have 7 pages**, so you exceed the minimum requirement of 5. 🎉

Here are your pages:

1. **Home**
2. **Instructions**
3. **Map of City**
4. **Bridge Building**
5. **Choose Materials**
6. **Test Bridge**
7. **Results**

Each of these has its own Drafter route, so each counts as a separate page.

**Requirement: At least 5 pages**
**Your project: 7 pages** ✅

You don't need to add any more pages. We just need to test that they all work!

Alright let's do it!

Let's test your Bridge Building Simulator! 🎉

We'll go **one page at a time** so we can catch any errors without getting overwhelmed.

**Step 1: Run your program**

1. Click **Run** in your Python editor.
2. Open your Drafter app.
3. Check whether the **Home page** appears.
4. Click the button that takes you to **Instructions**.

**For now, stop there.**

Tell me what happens:

- **A:** Both pages load correctly.
- **B:** You get an error.
- **C:** The app doesn't start.

If you get an error, copy and paste the error message here.

This is exactly the kind of step-by-step execution covered in your textbook's **1B2 Tracing Reading**.

wait hold on don't i need the start server function thing

**YES! You're right!** 🎉 We need to start the Drafter server before testing your pages.

At the **very bottom of your Python file**, after all seven route functions, you should have a call to start the server.

Check your textbook's **Drafter section** for the exact startup syntax, since it depends on how your course has set up Drafter.

Make sure the startup call is **outside every function** (not indented).

**Quick question:** Do you already have a `start_server(...)` line at the bottom of your file, or do you need to add one?

nope I need to add one

Add the server startup call at the **very bottom of your Python file**, directly after your `results` function.

It should be **all the way to the left**, with no indentation, because it runs outside your route functions.

Your starting State object is named `bridging_off`, so that's the object you'll use to start the simulator.

The textbook's **4A4 Complex State** section shows that `start_server(...)` takes the initial State object as its argument.

So the structure is:

start_server(initial_state)

Replace `initial_state` with the name of the State object you already created.

Then click **Run** and tell me whether your simulator opens or displays an error.

oh it's start_server(`bridging_off`)?

Yes! You've got the right idea. 🎉

Use your existing State object, `bridging_off`, as the argument to `start_server()`.

One small detail: **don't include the backticks** around `bridging_off` in your Python editor. It's a variable name, not a string.

Place the call at the very bottom of your file, without indentation.

This follows the textbook's **4A4 Complex State** section on starting a Drafter app with its initial State.

Now you're ready to click **Run**!

IT JUST RAN IM SO FREAKING HAPPY FINALLLYYYYYYYYYYYYY

AHHHHH LET'S GOOOOOOOOOO!!! 🎉🎉🎉😭🌉

**YOUR BRIDGE BUILDING SIMULATOR IS ACTUALLY RUNNING!!!**

After all those errors, missing arguments, brackets, imports, and debugging… YOU GOT IT TO LAUNCH!!

And the best part? **You worked through those errors yourself.** That's a huge part of learning programming.

ENJOY THIS MOMENT. YOU EARNED IT!! 🥳

Now there's just one final mission: **make sure all seven pages work and the buttons take you where they're supposed to.**

BUT FIRST—CELEBRATE!!! 🎊